Vision

Community-Driven Real-Time Incident Reporting

Software Requirements Specification | Hackathon MVP | 24-Hour Build

---

Version	1.0 — Hackathon Draft
Date	March 2026
Scope	24-Hour Hackathon MVP
Status	In Planning

1. Project Overview

1.1 Problem Statement

People are often unaware of incidents happening around them until it is too late. Information is fragmented across social media platforms, delayed through official emergency channels, or simply never reaches those who need it most. There is no single, real-time, community-driven source of truth for local incidents.

1.2 Solution

Vision is a community-driven map platform where users report incidents in real time. Anyone can pin an incident to the map with location, type, description, and a photo. All connected users see new reports appear instantly — no refresh required.

1.3 Key Value Propositions

- Real-time visibility: Incident markers appear on all connected users' maps the moment they are submitted
- Zero-friction reporting: Log in, click a location, fill a short form, done
- Community trust: Users can upvote or confirm existing reports to surface accurate information
- Actionable filtering: View only the incident types relevant to you (flood, crime, hazard, etc.)

Hackathon Goal

Deliver a functional, demo-ready MVP within 24 hours.

The core loop — submit a report, see it appear live on all connected maps — must work end-to-end.

Real-time functionality is a judging requirement and must be clearly demonstrable.

2. Scope

2.1 In Scope — Hackathon MVP

The following features are committed for the 24-hour build:

Feature	Description	Priority
User Authentication	Email/password login and registration via Firebase Auth or Supabase	Must Have
Interactive Map	Leaflet.js or Mapbox map with click-to-report functionality	Must Have
Incident Submission	Form capturing type, description, location (lat/lng), and optional photo	Must Have
Real-Time Markers	WebSocket broadcast pushes new markers to all connected clients instantly	Must Have
Marker Detail View	Click any marker to see incident details in a side panel or popup	Must Have
Filter by Type	Toggle buttons to show/hide markers by incident category	Should Have
Severity Level	Optional severity tag (Low / Medium / High) on each report	Should Have
Photo Upload	Attach a photo to a report; stored in cloud storage (Firebase/S3)	Should Have
Upvote / Confirm	Other users can upvote a report to signal it is still active	Nice to Have
Expiry Timer	Reports auto-expire after a configurable window (e.g., 6 hours)	Nice to Have

2.2 Out of Scope — Future Features

These ideas are valuable but are deferred beyond the hackathon:

- Push / browser notifications when an incident is reported near a user's saved location
- Native mobile app (iOS / Android)
- Moderator dashboard to review and remove false reports
- AI-powered duplicate detection to merge similar nearby reports
- Heatmap view aggregating incident density over time
- Integration with official emergency services APIs (911 dispatch, weather alerts)
- Verified responder accounts (police, fire, NGOs) with elevated credibility badges
- Historical analytics — trends, charts, export to CSV
- Multi-language support and accessibility audit
- Offline mode with sync when connectivity is restored

3. Functional Requirements

3.1 Authentication

- Users must register with an email and password
- Users must log in before submitting a report
- Unauthenticated users may view the map in read-only mode
- Session persists across page refreshes

3.2 Map Interface

- On load, the map centres on the user's current geolocation (with permission fallback to a default city)
- All active incident markers are fetched and rendered on map load
- Clicking any empty map location opens the incident submission form with the clicked coordinates pre-filled
- Markers use distinct icons per incident type (accident, hazard, crime, flood, other)
- Clicking a marker opens a detail panel/popup showing all report fields

3.3 Incident Reporting

The submission form collects the following fields:

Field	Type	Required	Notes
Incident Type	Dropdown select	**Yes**	Accident, Hazard, Crime, Flood, Other
Description	Text area (max 280 chars)	**Yes**	Short summary of the incident
Location	Lat/Lng (auto from map click)	**Yes**	Pre-filled; user can see on map
Photo	File upload (image)	No	Uploaded to cloud storage
Severity	Radio: Low / Med / High	No	Defaults to Medium if skipped
Timestamp	Auto-generated	Auto	Set server-side, not user input
User ID	Derived from auth session	Auto	Linked to logged-in user

3.4 Filtering

- A filter bar above the map shows a toggle chip per incident type
- Toggling a type hides/shows all matching markers immediately (client-side, no re-fetch)
- An 'All' toggle resets all filters

4. Real-Time Requirements (Judging Criterion)

Real-time functionality is a core hackathon requirement. The following section defines the expected behaviour and recommended implementation.

> **Why This Matters for Judging**
>
> Judges will explicitly evaluate real-time features. This means the demo must visibly show a report submitted in one browser window appearing on a second window with zero manual refresh.
>
> Plan your demo to have two browser windows side-by-side to showcase this clearly.

4.1 Real-Time Event Flow

1. User submits an incident report via the form
2. Frontend sends the report payload to the backend REST endpoint (POST /api/incidents)
3. Backend validates the payload and saves it to the database
4. Backend emits a WebSocket event (e.g., new_incident) to all connected clients, attaching the saved incident object
5. Each connected client receives the event and appends the new marker to the map in real time
6. User sees the marker appear without refreshing

4.2 WebSocket Implementation Options

Option	Description	Recommended For
Socket.io(Node.js)	Full-featured WebSocket library with automatic fallback to long polling. Client and server libraries available. Easiest to set up in 24 hours.	**Node / Express teams**
Supabase Realtime	Built-in Postgres change listener. When a row is inserted, Supabase pushes it to subscribed clients automatically — no custom WebSocket server needed.	**Teams using Supabase**
Firebase RTDB	Firebase Real-Time Database triggers client listeners on any data change. Zero server code required for real-time push.	**Teams using Firebase**

4.3 Socket.io Code Skeleton (Recommended)

Server (Node.js / Express):

```
io.on('connection', (socket) => {
```

```
  console.log('Client connected: ' + socket.id);
});
// After saving to DB:
io.emit('new_incident', savedIncident);
```

Client (React / Vanilla JS):

```
const socket = io(SERVER_URL);
socket.on('new_incident', (incident) => {
  addMarkerToMap(incident);  // your map function
});
```

5. Technical Architecture

5.1 Recommended Stack

Layer	Technology	Rationale
Frontend	React + Vite (or Next.js)	Fast setup, component model suits map + form UI
Map Library	Leaflet.js (free) or Mapbox GL	Leaflet requires no API key; Mapbox has a free tier
Backend	Node.js + Express	Easy WebSocket integration with Socket.io
Real-Time	Socket.io	Simplest full-duplex events in 24 hours
Database	Supabase (Postgres) or Firebase	Managed, free tier, built-in auth + storage
Auth	Supabase Auth or Firebase Auth	Email/password out of the box
File Storage	Supabase Storage or Firebase Storage	Paired with your DB choice
Hosting	Vercel (frontend) + Railway / Render (backend)	Deploy in minutes with Git push

5.2 API Endpoints

Method	Endpoint	Auth Required	Description
POST	`/api/auth/register`	No	Create a new user account
POST	`/api/auth/login`	No	Returns a session token
GET	`/api/incidents`	No	Fetch all active incidents
POST	`/api/incidents`	Yes	Submit a new incident report
GET	`/api/incidents/:id`	No	Fetch a single incident's details
POST	`/api/incidents/:id/upvote`	Yes	Upvote / confirm a report

5.3 Database Schema

incidents table

Column	Type	Description
id	`UUID / String`	Primary key, auto-generated
user_id	`String (FK)`	References the submitting user
type	`Enum`	accident | hazard | crime | flood | other
description	`Text`	Max 280 characters
latitude	`Float`	WGS84 decimal degrees
longitude	`Float`	WGS84 decimal degrees
photo_url	`String (nullable)`	Cloud storage URL
severity	`Enum`	low | medium | high
upvotes	`Integer`	Count of confirmations, default 0
expires_at	`Timestamp (nullable)`	Auto-expire time if set
created_at	`Timestamp`	Server-generated on insert

6. Suggested 24-Hour Build Plan

Below is a recommended timeline. Adjust based on team size and strengths.

Hours	Phase	Deliverables
00 – 02	**Setup & Architecture**	Repo created, stack decided, env variables configured, deployment pipelines set up
02 – 05	**Auth + Database**	User registration, login, DB schema created, basic CRUD for incidents
05 – 09	**Map + Reporting UI**	Map renders, click-to-report works, form submits to backend, markers load on page open
09 – 13	**Real-Time WebSockets**	Socket.io integrated, new reports broadcast to all clients, live demo working end-to-end
13 – 16	**Polish Core Features**	Marker icons per type, detail popup, filter bar, error handling, loading states
16 – 19	**Should-Have Features**	Photo upload, severity field, upvote button
19 – 22	**Testing & Bug Fixes**	Cross-browser test, fix crashes, ensure real-time works reliably on deployed URL
22 – 24	**Demo Prep**	Record a demo video backup, write a 2-minute pitch, prepare a live two-window demo

Critical Path Warning

If anything must slip, cut Nice-to-Have features first, never the real-time WebSocket integration.

A working Socket.io demo is worth more to judges than a polished UI without real-time.

Aim to have a working end-to-end real-time demo by Hour 13 at the latest.

7. Future Features (Post-Hackathon Roadmap)

The following features extend the product into a production-ready application after the hackathon.

7.1 Short-Term (1–4 Weeks)

- Push notifications — Browser or mobile alerts when a new report appears within a user-configured radius
- Report moderation — Flag a report as inaccurate; auto-hide if a threshold of flags is reached
- User profile page — View all reports submitted by a user, reputation score
- Improved map clustering — Group nearby markers at lower zoom levels to reduce visual clutter

7.2 Medium-Term (1–3 Months)

- Native mobile app — React Native (iOS + Android) with background location and push support
- Heatmap mode — Aggregate view showing incident density, filterable by time range
- AI duplicate detection — NLP + geofencing to identify and merge duplicate reports automatically
- Verified responder accounts — Official emergency services accounts with a trust badge
- Historical analytics dashboard — Charts, trends, and CSV export for researchers and journalists

7.3 Long-Term Vision

- Emergency services API integration — Two-way data sharing with 911 dispatch, weather alert systems, and NGO platforms
- Offline-first architecture — Reports queued locally and synced when connectivity returns
- Multi-language support — Localisation for the top 10 languages by population
- Accessibility audit — Full WCAG 2.1 AA compliance for inclusive use during emergencies
- Government / Smart City partnerships — Licensing the platform to municipalities as a public safety tool

8. Non-Functional Requirements

8.1 Performance

- Map should load all existing markers within 2 seconds on a standard broadband connection
- WebSocket event to new marker render should take under 500 ms end-to-end

8.2 Security (Hackathon Minimum)

- Passwords must never be stored in plain text (delegated to Firebase/Supabase Auth)
- Incident submission endpoints must reject unauthenticated requests with a 401 response
- Photo uploads must validate MIME type and reject non-image files

8.3 Reliability

- If a WebSocket connection drops, the client must attempt reconnection automatically (Socket.io handles this by default)
- On reconnect, the client re-fetches all incidents to catch any missed during the disconnect

9. Glossary

Term	Definition
WebSocket	A persistent, full-duplex communication protocol over TCP that allows servers to push data to clients in real time
Socket.io	A JavaScript library that wraps WebSockets and provides fallback transports, rooms, and event-based messaging
Marker	A visual pin on the map representing a submitted incident report
Upvote	A user action confirming that a reported incident is accurate and still ongoing
Expiry	An automatic expiration applied to a report after a set duration to keep the map current
Supabase	An open-source Firebase alternative built on PostgreSQL, with built-in auth, storage, and real-time subscriptions
Leaflet.js	An open-source JavaScript library for interactive maps, requiring no paid API key

End of Document

Vision — Hackathon SRS v1.0